

Training_performance_plotter.py Documentation

PyQt6-Based Training Monitoring and Visualization Tool

Updated: 2026-03-01

Warning: This documentation was generated by an AI assistant. However I have reviewed this file and made changes accordingly if needed.

Abstract

`Training_performance_plotter.py` is a PyQt6-based GUI application for monitoring transformer training performance. It provides real-time resource usage monitoring (CPU, GPU, RAM), linguistic performance metrics (BLEU, CER, WER), attention heatmap visualization, and process hierarchy tree views with support for epoch-based filtering and parallel process visualization.

Contents

1 Overview

The `Training_performance_plotter.py` module implements a comprehensive training monitoring system with four main tabs:

- **Performance Reports:** Real-time CPU, GPU, and RAM usage with epoch-based filtering
- **Linguistic Performance Report:** BLEU, CER, and WER metrics over epochs
- **Performance During Inference:** Attention heatmaps and inference resource usage
- **Process Mindmap:** Hierarchical tree view of process relationships with parallelism detection

2 Dependencies

The module requires the following Python packages:

```
1 import sys
2 import pandas as pd          # Data processing
3 import numpy as np          # Numerical operations
4 from pathlib import Path     # File path handling
5 import psutil               # System resource monitoring
6 from PyQt6 import QtCore, QtWidgets, QtGui # GUI framework
7 import pyqtgraph as pg      # High-performance plotting
8 import torch                # PyTorch integration
9 from torchmetrics.text.bleu import BLEUScore # BLEU metric calculation
10 import threading            # Threading for inference sampling
11 import time                 # Timing operations
12 import math                 # Mathematical functions
```

Additional internal dependencies:

- `translate.py`: Translation inference functions
- `train.py`: Model loading and dataset functions
- `config.py`: Configuration and epoch retrieval
- `processes.py`: Process hierarchy and JSON storage

3 Class Reference

3.1 ResourceMonitorApp

The main application class extending `QtWidgets.QMainWindow`.

3.1.1 Constructor

```
1 class ResourceMonitorApp(QtWidgets.QMainWindow):
2     def __init__(self):
3         # Initialize data paths
4         self.log_dir = Path("eval_results/resource_usage")
5         self.file_sec = self.log_dir / "usage_seconds.csv"
6
7         # Initialize state flags
8         self._live_mode = False          # Live monitoring enabled
```

```

9     self._data_loaded = False           # Data has been selected
10    self._epoch_range = (None, None)    # Selected epoch range
11    self._cached_df = None               # DataFrame cache

```

3.1.2 Key Attributes

Attribute	Type	Description
file_sec	Path	Path to per-second resource usage CSV
timer	QTimer	1-second timer for live updates
_live_mode	bool	Live mode state flag
_epoch_range	tuple	Selected (start_epoch, end_epoch)
_cached_df	DataFrame	Cached data for performance

3.2 Data Loading and Processing

3.2.1 CSV Data Format

The per-second resource usage file (`usage_seconds.csv`) format:

```

1 time,cpu_percent,ram_gb,gpu_gb,gpu_total_gb,epoch_marker
2 1762145228.85,0.0,7.49,0.0,5.67,
3 1762145229.85,12.7,7.44,0.0,5.67,epoch_0
4 1762145230.85,19.6,7.46,0.0,5.67,

```

Columns:

- `time`: Unix timestamp (seconds since epoch)
- `cpu_percent`: CPU utilization percentage
- `ram_gb`: RAM usage in gigabytes
- `gpu_gb`: GPU memory usage in gigabytes
- `gpu_total_gb`: Total GPU memory available
- `epoch_marker`: Optional marker indicating epoch start (e.g., “epoch_5”)

3.2.2 Epoch Extraction

The plotter extracts epoch information from the `epoch_marker` column:

```

1 if "epoch_marker" in df.columns:
2     df["epoch"] = df["epoch_marker"].fillna("").apply(
3         lambda x: int(x.split("_")[1])
4         if isinstance(x, str) and x.startswith("epoch_")
5         else np.nan
6     )
7     # Forward-fill epoch values for all data points
8     df["epoch"] = df["epoch"].fillna(method="ffill")
9     df["epoch"] = df["epoch"].fillna(method="bfill")

```

3.3 Plot Update Methods

3.3.1 update_pReport_plots()

Main update method for Performance Reports tab:

```
1 def update_pReport_plots(self):
2     """Update per-second resource usage plots."""
3     # Check if data should be updated
4     file_stat = self.file_sec.stat()
5     cache_key = (file_stat.st_mtime, file_stat.st_size)
6
7     if (self._live_mode or self._cached_df is None or
8         getattr(self, '_cache_key', None) != cache_key):
9         df = pd.read_csv(self.file_sec)
10        # ... process and cache data
```

3.3.2 Live Mode Filtering

```
1 def _get_live_window_range(self, df):
2     """Filter DataFrame based on live window selection."""
3     window_value = self.live_window_combo.currentData()
4
5     if window_value > 0:
6         # Time-based window (seconds)
7         min_time = max(0, df["time"].max() - window_value)
8         return df[df["time"] >= min_time]
9     else:
10        # Epoch-based window
11        num_epochs = abs(window_value)
12        sorted_epochs = sorted(df["epoch"].dropna().unique())
13        start_epoch = sorted_epochs[-num_epochs]
14        return df[df["epoch"] >= start_epoch]
```

4 User Interface

4.1 Performance Reports Tab

4.1.1 Control Panel

- **Live Mode Toggle:** Enable/disable auto-updating during training
- **Window Selector:** Choose time window (60s, 5min, 15min) or epoch window (1, 3 epochs)
- **Start/End Epoch:** Manual epoch range selection
- **Apply Range:** Apply selected epoch filter
- **Show All:** Reset to show all epochs
- **Sampling Dropdown:** Data downsampling ratio (1:1, 1:5, 1:10, etc.)
- **Print to PDF:** Export current view to PDF

4.1.2 Per-Second Resource Usage Plots

Three time-series plots showing high-resolution resource usage:

- **CPU Usage:** Percentage utilization over time
- **GPU Memory:** GB allocated over time
- **RAM Memory:** GB used over time

Features:

- Epoch boundary lines (vertical dashed lines)
- Interactive zoom and pan
- Point count display with sampling indicator
- Min-max downsampling for large datasets

4.1.3 Average Per-Epoch Plots

Aggregated metrics by epoch:

- Average CPU usage per epoch
- Average GPU memory per epoch
- Average RAM usage per epoch

4.2 Linguistic Performance Report Tab

Displays translation quality metrics from `eval_metrics.json`:

- **CER** (Character Error Rate): Red line with circles
- **WER** (Word Error Rate): Blue line with circles
- **BLEU** Score: Green line with circles

Data source format:

```
1 [
2   {"epoch": 0, "cer": 10.72, "wer": 7.67, "bleu": 0.0},
3   {"epoch": 45, "cer": 0.06, "wer": 0.36, "bleu": 0.85}
4 ]
```

4.3 Performance During Inference Tab

4.3.1 Model Loading

```
1 def load_model_and_generate_data(weights_path: str,
2                                   gpu_sample_interval: float = 0.1):
3     # Load model and weights
4     config = get_config()
5     model = get_model(config, ...).to(device)
6     state = torch.load(weights_path, map_location=device)
7     model.load_state_dict(state['model_state_dict'])
8
```

```

9      # Sample GPU/RAM during inference
10     decode_thread = threading.Thread(target=run_decode)
11     decode_thread.start()
12
13     while decode_thread.is_alive():
14         gpu_gb = torch.cuda.memory_allocated(device) / 1e9
15         ram_samples.append((time.time() - t0,
16                             psutil.virtual_memory().used / 1024**3))
17         time.sleep(gpu_sample_interval)

```

4.3.2 Attention Heatmaps

Generates 18 attention visualizations (6 layers $\times$ 3 types):

- Encoder Self-Attention
- Decoder Self-Attention
- Encoder-Decoder Cross-Attention

Each heatmap shows token-to-token attention weights with:

- Custom blue-green color map
- Token labels on axes
- Color bar indicating attention strength

4.4 Process Mindmap Tab

Visualizes process hierarchy from `eval_results/processes.json`.

4.4.1 UID Format Parsing

The new UID format includes epoch and layer information:

```

1 def get_epoch_from_uid(uid):
2     """Extract epoch from UID format: epochxlayer-id."""
3     if "x" in uid and "-" in uid:
4         return int(uid.split("x")[0]) # "5x2-abc" -> 5
5     return 0
6
7 def get_layer_from_uid(uid):
8     """Extract layer from UID format: epochxlayer-id."""
9     if "x" in uid and "-" in uid:
10        layer_part = uid.split("x")[1] # "5x2-abc" -> "2-abc"
11        return int(layer_part.split("-")[0]) # -> 2
12    return 0

```

4.4.2 Parallelism Detection

Processes with the same parent and same layer are marked as parallel:

```

1 # Group subtasks by layer
2 for subtask_uid in subtasks:
3     st_layer = get_layer_from_uid(subtask_uid)
4     if st_layer not in subtask_layers:
5         subtask_layers[st_layer] = []

```

```

6     subtask_layers[st_layer].append(subtask_uid)
7
8 # Mark groups with multiple processes as parallel
9 parallel_groups = {layer: tasks for layer, tasks in
10                    subtask_layers.items() if len(tasks) > 1}

```

Tree columns:

- **Process:** Name with UID ([P] prefix for parallel)
- **Epoch:** Training epoch number
- **Layer:** Parallelism layer
- **Duration:** Execution time
- **Execution:** Sequential or Parallel

5 Data Downsampling

5.1 Min-Max Downsampling

For large datasets, min-max downsampling preserves peaks and valleys:

```

1 def minmax_downsample(x, y, target_points=2000):
2     """Keep min and max in each bin to preserve extrema."""
3     n = len(x)
4     if n <= target_points:
5         return x, y
6
7     num_bins = max(1, target_points // 2)
8     bin_size = max(2, n // num_bins)
9
10    x_down, y_down = [], []
11    for i in range(num_bins):
12        start = i * bin_size
13        end = min((i + 1) * bin_size, n)
14        bin_x, bin_y = x_arr[start:end], y_arr[start:end]
15
16        # Find indices of min and max
17        min_idx = np.argmin(bin_y)
18        max_idx = np.argmax(bin_y)
19
20        # Add both points (sorted by x to maintain order)
21        if bin_x[min_idx] <= bin_x[max_idx]:
22            x_down.extend([bin_x[min_idx], bin_x[max_idx]])
23            y_down.extend([bin_y[min_idx], bin_y[max_idx]])
24        else:
25            x_down.extend([bin_x[max_idx], bin_x[min_idx]])
26            y_down.extend([bin_y[max_idx], bin_y[min_idx]])
27
28    return np.array(x_down), np.array(y_down)

```

5.2 Uniform Sampling

User-selectable sampling rates (1:1, 1:5, 1:10, etc.):

```
1 def _apply_sampling(self, x, y, sampling_rate):
2     """Apply uniform sampling to data."""
3     x_arr = x.values if hasattr(x, 'values') else np.array(x)
4     y_arr = y.values if hasattr(y, 'values') else np.array(y)
5
6     if sampling_rate <= 1:
7         return x_arr, y_arr
8
9     n = len(x_arr)
10    if n <= sampling_rate:
11        return x_arr, y_arr
12
13    indices = np.arange(0, n, sampling_rate)
14    if indices[-1] != n - 1:
15        indices = np.append(indices, n - 1)
16
17    return x_arr[indices], y_arr[indices]
```

6 PDF Export

Three tabs support PDF export using Qt's QPdfWriter:

```
1 def _print_performance_report_to_pdf(self):
2     file_path, _ = QtWidgets.QFileDialog.getSaveFileName(
3         self, "Save Performance Report as PDF",
4         str(Path.cwd() / "performance_report.pdf"),
5         "PDF Files (*.pdf)"
6     )
7
8     if file_path:
9         pdf_writer = QtGui.QPdfWriter(file_path)
10        pdf_writer.setPageLayout(QPageLayout(
11            QPageSize(QPageSize.PageSizeId.A4),
12            QPageLayout.Orientation.Portrait,
13            QtCore.QMarginsF(20, 20, 20, 20)
14        ))
15
16        painter = QPainter(pdf_writer)
17        widget = self.scroll_pReport.widget()
18        widget.render(painter) # Render scroll area to PDF
19        painter.end()
```

Multi-page support for inference tab (attention heatmaps can span multiple pages).

7 Usage

7.1 Starting the Application

```
1 python Training_performance_plotter.py
```

7.2 Viewing Live Training

1. Start training: `python train.py`
2. Open Performance Plotter

3. Go to “Performance Reports” tab
4. Check “ LIVE MODE” checkbox
5. Select window size (e.g., “Last 5 min”)
6. Plots will auto-update every second

7.3 Viewing Historical Data

1. Enter epoch range in Start/End fields
2. Click “Apply Range”
3. Or click “Show All” to see complete history

7.4 Viewing Attention Maps

1. Go to “Performance During Inference” tab
2. Click “Select Weights File (.pt)”
3. Choose a trained model checkpoint
4. Click “Generate Attention Maps”
5. Wait for model load and inference

8 File Locations

File	Description
eval_results/resource_usage/usage_seconds.csv	Per-second resource usage data
eval_results/resource_usage/usage_epochs.csv	Per-epoch aggregated data
eval_results/eval_metrics.json	BLEU, CER, WER metrics
eval_results/processes.json	Process hierarchy data
weights/*.pt	Model checkpoint files

9 Notes and Limitations

- **File Locking:** CSV files may be locked if open in spreadsheet applications
- **Memory Usage:** Large CSV files (>100MB) may cause UI lag without downsampling
- **GPU Monitoring:** Requires NVIDIA GPU with CUDA support
- **Live Mode:** Only updates when training is actively writing to CSV
- **Epoch Markers:** Require proper epoch marking in training script
- **Parallel Detection:** Based on layer field in process data